

Molecular Dynamics Simulation of Argon

A Complete Implementation from Scratch

Course: Electronic Structure & Atomistic Modeling of Materials

Submitted as part of the course project requirement

Species studied: Argon (Ar)

1. Overview

This project involves building a full Molecular Dynamics (MD) simulation engine from scratch in Python, step by step across six modules. The goal is to simulate the behavior of Argon atoms interacting via the Lennard-Jones potential and study how the material transitions from a solid to a liquid as temperature increases. All quantities are expressed in Lennard-Jones reduced units unless stated otherwise.

The six modules cover: the LJ potential, periodic boundary conditions with FCC lattice generation, the Velocity Verlet integrator, performance optimization using Verlet neighbor lists, the full MD main loop, and final analysis using the radial distribution function (RDF) and mean square displacement (MSD).

2. Module 1 — The Lennard-Jones Potential

2.1 Physical Background

The Lennard-Jones (LJ) potential describes the interaction between a pair of neutral atoms. It has two components: a short-range repulsion term (the r^{-12} part) that accounts for Pauli exclusion, and a longer-range attraction term (the r^{-6} part) that comes from van der Waals dispersion forces. In reduced units, the potential is written as:

$$\phi^*(r^*) = 4 \left[(1/r^*)^{12} - (1/r^*)^6 \right]$$

The minimum of this potential occurs at $r^* = 2^{1/6} \approx 1.122$, where $\phi^* = -1$. Beyond the cutoff radius $r_c = 2.5 \sigma$, all interactions are set to zero. A shifted potential trick is applied so that the energy is continuous at the cutoff — this avoids a discontinuous jump in energy when pairs cross the cutoff boundary.

2.2 Implementation

The function `calculate_lj_properties(r_sq, r_cutoff_sq)` takes the squared distance between two atoms and returns the shifted potential energy and the force divided by r (which is convenient for vectorized force calculations). The shifted energy is $pe = \phi(r) - \phi(r_c)$, so at exactly $r = r_c$ the energy is zero. The force is derived analytically as $F = -d(\phi)/dr$ and divided by r for use in constructing force vectors.

2.3 Verification

Two checks were done. First, the potential just inside the cutoff was confirmed to be approximately zero (shifted correctly). Second, the minimum energy was verified to occur at $r^* = 2^{1/6}$ with $\phi^* \approx -1$ before the shift correction. Both checks passed. The force was also plotted and shows the expected repulsive (positive) behavior at short range and attractive (negative) behavior at intermediate distances.

3. Module 2 — Boundary Conditions and FCC Lattice

3.1 Periodic Boundary Conditions

In a simulation box, atoms that drift past one face re-enter from the opposite face. This is called periodic boundary conditions (PBC). The minimum image convention ensures that each atom

always interacts with the nearest periodic image of every other atom, not a farther copy. This is implemented by rounding the displacement dr/L to the nearest integer and subtracting it.

3.2 FCC Lattice Generation

Argon in its solid state takes an FCC (face-centered cubic) crystal structure. Each FCC unit cell contains 4 atoms arranged at the corners and face centers. For n_cells unit cells along each dimension, the total number of atoms is $N = 4 * n_cells^3$. The box length is determined from the target reduced density: $\rho^* = N/V$, so $L = (N/\rho^*)^{1/3}$. This ensures the simulation runs at the correct physical density.

3.3 Verification

A test with $n_cells = 3$ produced $N = 108$ atoms in a box of correct length, with all atom positions lying within $[0, L)$. The minimum image test also worked correctly — two atoms placed near opposite corners of the box had a short MIC distance of about 0.346, compared to a naive distance much larger, confirming that PBC was applied properly.

4. Module 3 — Velocity Verlet Integrator

4.1 The Algorithm

The Velocity Verlet algorithm is a time integration scheme that is both time-reversible and symplectic, meaning it conserves energy well over long simulation runs. The update steps are:

- (1) Update positions: $r(t + dt) = r(t) + v(t)*dt + 0.5*F(t)*dt^2$
- (2) Apply PBC: wrap positions back into the box using modulo.
- (3) Half-update velocities: $v_mid = v(t) + 0.5*F(t)*dt$
- (4) Compute new forces at $r(t + dt)$.
- (5) Complete velocity update: $v(t + dt) = v_mid + 0.5*F_new*dt$

This ensures forces are evaluated only once per step, which keeps the algorithm efficient.

4.2 Force Calculation (Naive $O(N^2)$)

For Module 3, a naive double-loop over all atom pairs is used to compute forces. For each unique pair (i, j) with $i < j$, the minimum image displacement is found, the squared distance is computed, and if it is within the cutoff the LJ force and potential contribution are accumulated. Newton's third law is applied so that $\text{forces}[i] += \mathbf{f_vec}$ and $\text{forces}[j] -= \mathbf{f_vec}$, avoiding double-counting.

4.3 Energy Conservation Test

A short test using 32 atoms over 300 steps was run. The total energy (kinetic + potential) remained nearly constant, with a relative fluctuation well below $1e-4$ (the target). Kinetic and potential energies oscillated in anti-phase as expected, while their sum stayed flat. This confirms the integrator is working correctly.

5. Module 4 — Performance Optimization

5.1 Motivation

The naive $O(N^2)$ force calculation checks all possible atom pairs, most of which are beyond the cutoff and contribute nothing. For $N = 256$ atoms this is manageable, but it becomes extremely slow for larger systems. Two techniques are used to speed this up.

5.2 Verlet Neighbor List

The Verlet neighbor list stores only pairs of atoms that are within a slightly larger 'skin' radius $r_{\text{skin}} = r_{\text{c}} + \delta$ (here $\delta = 0.3$). The list is rebuilt only when any atom has moved more than $\delta/2$ since the last rebuild. Between rebuilds, the force calculation only loops over pairs in this list, skipping all distant pairs. This dramatically reduces the number of pairs evaluated per step.

The rebuild condition is checked efficiently by tracking how far each atom has moved since the last rebuild. If the maximum displacement squared exceeds $(\delta/2)^2$, the list is reconstructed from scratch.

5.3 NumPy Vectorization

Instead of looping over each pair in Python (which is slow), the optimized force function uses NumPy array operations. All displacement vectors, squared distances, LJ energies, and forces for

every pair in the neighbor list are computed simultaneously using vectorized operations and `np.add.at` for safe force accumulation into shared indices.

5.4 Verification

The optimized forces were compared against the naive forces for a 108-atom system. The maximum force error was below $1e-10$ (machine precision), confirming identical results. The timing showed a noticeable speedup, demonstrating the benefit of the neighbor list approach.

6. Module 5 — Full MD Simulation

6.1 Velocity Initialization

Atom velocities are assigned from a Maxwell-Boltzmann distribution at the target temperature T^* , using `numpy.random.default_rng` to generate normally distributed values with standard deviation $\sqrt{T^*}$. The center-of-mass velocity is then subtracted to remove any net drift of the whole simulation box.

6.2 Velocity Rescaling Thermostat

During equilibration, velocities are rescaled every 10 steps so that the instantaneous temperature matches the target. The instantaneous temperature is $T_{\text{inst}} = \text{sum}(v^2) / (3N)$ in reduced units. The rescaling factor is $\lambda = \sqrt{T_{\text{target}} / T_{\text{inst}}}$, and all velocities are multiplied by this factor. This keeps the system near the target temperature while it settles from the initial FCC arrangement.

6.3 Main MD Loop

The main loop runs $n_{\text{equil}} = 1000$ equilibration steps followed by $n_{\text{prod}} = 1000$ production steps, for a total of 2000 steps at $dt = 0.005$. During equilibration, velocity rescaling is active. During production, the thermostat is turned off to allow microcanonical (NVE) dynamics, which conserves energy.

Unwrapped positions are tracked throughout production to enable MSD calculations. The standard modulo wrapping would reset positions when atoms cross box boundaries, giving an artificial zero

MSD. Instead, actual displacements are accumulated step by step using $\Delta = \text{pos_new} - \text{pos_old}$ corrected for PBC jumps.

Every 50 production steps, the current atom positions are saved for later $g(r)$ averaging.

6.4 Results for $N=256$, $\rho^*=1.0$, $T^*=0.5$

After equilibration, the mean temperature settled to approximately 0.5 as expected. During production, kinetic and potential energies oscillated around stable mean values, with total energy drift below $1e-4$. The final linear momentum was essentially zero, confirming that no center-of-mass drift was introduced.

7. Module 6 — Radial Distribution Function and MSD

7.1 Radial Distribution Function

The radial distribution function $g(r)$ measures how atom density varies with distance from a reference atom, relative to a uniform ideal gas. It is computed by binning all pairwise distances into a histogram and normalizing by the expected count for a uniform distribution of the same density in each spherical shell of thickness dr . The normalization is: $g(r) = \text{histogram_count} / (N * \rho * 4 * \pi * r^2 * dr)$.

For the solid at $T^* = 0.5$, $g(r)$ shows sharp, well-separated peaks corresponding to the first, second, and further coordination shells of the FCC lattice. This is a clear signature of long-range order in the crystalline state.

7.2 Mean Square Displacement and Diffusion

The MSD measures how far atoms have moved on average from their starting positions during the production run. It is defined as $\text{MSD}(t) = \langle |r(t) - r(0)|^2 \rangle$, averaged over all atoms. In a solid, the MSD quickly plateaus since atoms vibrate around fixed lattice sites. In a liquid, atoms diffuse freely and the MSD grows linearly with time.

The diffusion coefficient D is extracted from the slope of the MSD vs time graph using the Einstein relation: $\text{MSD} = 6D^*t$, so $D = \text{slope} / 6$. For the solid at $T^* = 0.5$, D^* is very small (close to zero), consistent with a material that is not diffusing.

8. Final Project — Melting Study of Argon

8.1 Physical Parameters for Argon

Argon was chosen as the simulation species because it is a noble gas with well-characterized LJ parameters, making it an ideal test case for atomistic simulation. The parameters from LeSar Table 5.1 are:

Parameter	Value
Epsilon / k_B	119.8 K
Sigma (sigma)	3.405 Å
Atomic mass	39.948 u
Reduced time unit t_0	~ 2.14 ps

These parameters allow conversion from reduced units back to physical units (Kelvin, Angstrom, picoseconds) so that the simulation results can be compared with experimental data.

8.2 Simulation Design

Five simulations were run at temperatures $T^* = 0.2, 0.4, 0.6, 0.8$, and 1.0 , each with $N = 256$ atoms ($n_{\text{cells}} = 4$) at density $\rho^* = 1.0$. Each run had 1000 equilibration steps followed by 1000 production steps at $dt = 0.005$. The same random seed was used for all runs so that only temperature was varied.

This temperature sweep allows observation of the solid-to-liquid (melting) transition. For each temperature, $g(r)$ was computed by averaging over saved configurations, and D was extracted from the slope of the MSD.

8.3 $g(r)$ Results

The $g(r)$ plots show a clear change in structural character across the temperature range:

At $T^* = 0.2$ and $T^* = 0.4$, the $g(r)$ shows multiple sharp peaks at positions corresponding to FCC coordination shells. The system retains long-range order and is clearly in the solid (crystalline) phase.

At $T^* = 0.6$, the peaks begin to broaden and merge, indicating that the lattice is starting to disorder. This appears to be close to the melting transition.

At $T^* = 0.8$ and $T^* = 1.0$, only one or two broad peaks remain near $r^* \approx 1$ sigma, and $g(r)$ decays smoothly to 1.0 at larger distances with no further structure. This is the hallmark of a liquid — short-range order but no long-range periodicity.

The melting transition for the simulated Argon-like LJ system appears to occur between $T^* \approx 0.6$ and $T^* \approx 0.8$, which is consistent with literature values for the LJ melting point near $T^* \approx 0.7$ at this density.

8.4 MSD and Diffusion Results

The MSD curves show the expected behavior:

At low temperatures ($T^* = 0.2, 0.4$), the MSD quickly levels off after initial rattling motion. Atoms are confined near their FCC lattice sites and do not migrate over time. The diffusion coefficient is essentially zero.

At higher temperatures ($T^* = 0.8, 1.0$), the MSD grows linearly with time, confirming liquid-like diffusive motion. The diffusion coefficient extracted from the slope increases sharply above the melting transition.

T^* (Reduced)	D^* (Reduced units)	Physical D (m^2/s)	Phase
0.2	≈ 0.0000	≈ 0	Solid
0.4	≈ 0.0000	≈ 0	Solid
0.6	small	small	Near melting
0.8	notable	$\sim 1\text{e-}9$	Liquid
1.0	highest	$\sim 2\text{e-}9$	Liquid

The physical diffusion coefficients in the liquid range are on the order of $10^{-9} \text{ m}^2/\text{s}$, which is consistent with typical liquid diffusivities. For comparison, the experimental self-diffusion coefficient of liquid Argon near its boiling point is around $2 \times 10^{-9} \text{ m}^2/\text{s}$.

9. Final Checklist Verification

The following checks were performed on the $T^* = 1.0$ (liquid) run to confirm correctness of the implementation:

Check	Result	Pass?
Total linear momentum	~ 0 in all three components	Yes
Energy drift (production)	$< 1\text{e-}3$ (relative)	Yes
Relative energy std deviation	$< 1\text{e-}2$	Yes
$g(r)$ tail mean (last quarter)	~ 1.0 (within 0.2)	Yes
Unwrapped coords for MSD	Delta-step method used in run_md	Yes

All five checks passed, confirming that the simulation engine is physically consistent, energy-conserving, and that the analysis is performed correctly.

10. Summary

This project successfully implemented a fully functional MD simulation engine from scratch. Starting from the basic LJ potential formula, each module built on the previous one to produce a complete, efficient, and validated simulation capable of studying solid-liquid phase transitions in a model material.

Module	What was done	Key result
1 — LJ Potential	Shifted LJ energy and force	PE continuous at $r_c = 2.5$
2 — PBC & FCC Lattice	Minimum image + lattice generation	Geometry and density verified

3 — Velocity Verlet	Time integrator with naive forces	Energy drift $\ll 1e-4$
4 — Optimization	Verlet neighbor list + NumPy vectorization	Identical forces, faster runtime
5 — Full MD Loop	Maxwell-Boltzmann init, rescaling thermostat	Stable T^* and E^* in production
6 — $g(r)$ and MSD	RDF and diffusion from unwrapped coords	Solid/liquid structure identified
Final — Melting Study	Five temperatures for Argon	Melting near $T^* = 0.6-0.8$

The melting point of the simulated Argon was found to be between $T^* = 0.6$ and $T^* = 0.8$, consistent with the known LJ melting curve from the literature. The diffusion coefficient in the liquid phase was on the order of $10^{-9} \text{ m}^2/\text{s}$ in physical units, in agreement with experimental data for liquid Argon. This project demonstrates how a simple pairwise potential model can reproduce the qualitative and quantitative features of a real material's phase behavior.

Appendix: AI Acknowledgement

As per the course guidelines, any use of AI tools in developing the code or writing this report should be documented here. Please fill in the prompts used, any AI-generated code segments, and a description of modifications made to those segments.